Stage 2 Implementation Report

AI-powered Study Planner · EECS 3311 · Rafat Kabir
Living implementation report — F01 increment, October 31, 2026 (Toronto).

> This is a progress report, not a claim that Stage 2 is complete. F01 is deterministic. AI integration, the other proposed features, five course design patterns, feature videos, and weekly commit evidence remain outstanding. Update this source and regenerate `stage2-report.pdf` as implementation progresses.

1. Project overview and implementation status

The application helps students organize courses and eventually generate personalized study resources and plans. The current increment implements course creation and an owned-course dashboard. A Vaadin Flow Java UI runs in `frontend/`; a separate Spring Boot REST application runs in `backend/`. A Java 17 source-file CLI calls the same REST API. JDBC writes courses to a file-backed H2 database, so saved courses survive backend restarts.

The Stage 1 references are Task 1.2 F01, Task 2.2 UC-01, Tasks 3/4 F01, the Course/User UML classes, and SD01. The Stage 2 assignment requires traceability, GUI and CLI, ten meaningful features, five distinct meaningful course design patterns, an LLM with agent behavior, ongoing code commits, a PDF/Word report, demonstrations, and local-only credentials. Only the supplied Stage 1 report and Stage 2 instructions were available; any additional baseline project instructions must be checked before final submission.

Current limitations: one locally configured student account; no account registration or production identity provider; no edit/delete operations; no learning materials; no LLM, RAG, or planning agent yet. Authentication is supporting infrastructure and is not counted as a meaningful feature. Basic authentication is for local development; HTTPS is required for deployment outside localhost. The frontend retains its API credentials in server memory for the signed-in view. A page reload requires sign-in again. No credentials are put in browser localStorage.

2. Feature-to-implementation mapping

Field	F01 implementation
ID / Stage 1 reference	F01 — Course Creation; Task 1.2; UC-01 Manage Courses; SD01 (creation branch); Task 4 F01.

Status and type	Implemented creation/list/read flow; deterministic. “Manage” is scoped to creation and viewing in this increment. Editing/deleting remain unimplemented and are not claimed.
Inputs and rules	Name, description, start date, end date required. Name ≤120 characters; description ≤2000; whitespace-only values rejected; text trimmed. End must be on or after start. Same-day courses and duplicate names allowed: Stage 1 specified neither a minimum duration nor uniqueness. Dates use ISO YYYY-MM-DD over HTTP.
GUI	`frontend/.../views/MainView.java`: sign-in, dashboard, create dialog, required/invalid-field feedback, cancellation, refresh, saved course cards, safe API/network errors. Key methods: `showSignIn`, `showDashboard`, `openCourseForm`, `refreshCourses`, `renderCourses`. `themes/study-planner/styles.css`: responsive visual styling.
Frontend API boundary	`frontend/.../course/CourseApi.java`: authenticated HTTP calls and per-view cookie session; `viewCourses`, `createCourse`. `CourseApiFactory.connect` creates isolated clients. `Course` and `CreateCourseRequest` are transport records. This small factory is not claimed as the Stage 1 study-resource Factory pattern.
REST entry	`backend/.../course/CourseController.java`: `createCourse` (POST /api/courses), `viewCourses` (GET /api/courses), `getCourse` (GET /api/courses/{id}). The successful POST returns 201, the saved object, and its Location.
Application/domain	`CourseService` coordinates requests and transactions. `User.authenticated` derives a stable UUID from the authenticated username; `User.createCourse` creates the owned domain object; `User.viewCourses` queries storage. Immutable `Course` validates its invariants, dates, and ownership identity. `CreateCourseRequest` performs HTTP field validation.
Persistence	`CourseRepository` is the storage boundary; `JdbcCourseRepository.save/findByOwner/findByIdAndOwner` implements it using prepared JDBC statements. `schema.sql` defines the courses table, owner index, and date constraint. H2 data is ignored under `backend/data/`.
Failure and identity handling	`SecurityConfiguration` authenticates the configured student with BCrypt and retains CSRF protection; `CsrfController.token` provides a session token. `ApiExceptionHandler` returns safe validation, parsing, not-found, and storage errors. No owner ID is accepted in create input. List/read queries are owner-filtered. Unauthenticated requests fail; a cross-owner read returns 404.
CLI	`cli/StudyPlannerCli.java`: `main/run` accept `list`, `create`, and `help`; the CLI securely prompts for a password or reads the environment, obtains CSRF/cookies, calls the same API, and prints real JSON. Errors exit nonzero.
Patterns / video	No Stage 1 AI/resource pattern applies to F01 (N/A in Stage 1 mapping). Repository and layered separation support the feature but are not counted toward the five required course patterns. Video: pending owner recording; see Section 7.

Collaboration: Student signs in → MainView loads owned courses through CourseApi → student opens Create Course and submits four fields → UI validates → HTTP Basic, session CSRF, and request validation run in backend → CourseController calls CourseService → User.createCourse constructs a valid Course owned by the principal → JdbcCourseRepository inserts it transactionally → 201 returns

→ dialog closes and dashboard reloads the saved list. The CLI joins at the same HTTP boundary. A failure leaves the entered GUI fields available for correction; it never fabricates a saved card.

Stage 1 feature	Current status / implementation mapping	Planned Stage 1 participants and patterns	Video
F02 Upload Learning Materials	Not implemented; no upload controls or empty placeholder classes claimed.	Course, LearningMaterial, CourseController; N/A.	Pending
F03 Document Summarization	Not implemented.	LearningMaterial, StudyAssistantFacade, ContentAgent, AIEngine, StudyResourceFactory, Summary; Facade/Singleton/Factory.	Pending
F04 Concept Extraction	Not implemented.	ContentAgent, AIEngine, ConceptSet, StudyResourceFactory, StudyAssistantFacade; Facade/Singleton/Factory.	Pending
F05 Question Answering Over Materials	Not implemented.	CourseChat, ContentAgent, StudyAssistantFacade, AIEngine; Facade/Singleton.	Pending
F06 Personalized Study-Plan Generation	Not implemented.	StudyProfile, PlanningAgent, StudyPlan, StudySession, Calendar, AIEngine, factory/facade; Facade/Singleton/Factory/Observer.	Pending
F07 Flashcard Generation	Not implemented.	AssessmentAgent, FlashcardDeck, AIEngine, facade/factory; Facade/Singleton/Factory.	Pending
F08 Quiz Generation	Not implemented.	AssessmentAgent, Quiz, AIEngine, facade/factory; Facade/Singleton/Factory.	Pending
F09 Automatic Quiz Grading	Not implemented.	QuizGradingService, GradingStrategy, deterministic/AI strategies, GradeResult, AIEngine; Strategy/Singleton.	Pending
F10 Calendar	Not implemented.	Calendar, ScheduleSubject, ScheduleObserver, Course, StudyPlan, Quiz; Observer.	Pending
UC-11 Maintain Study Profile	Not implemented; supporting use case for F06, not silently counted as an extra feature.	StudyProfile, planning/calendar flow.	Pending

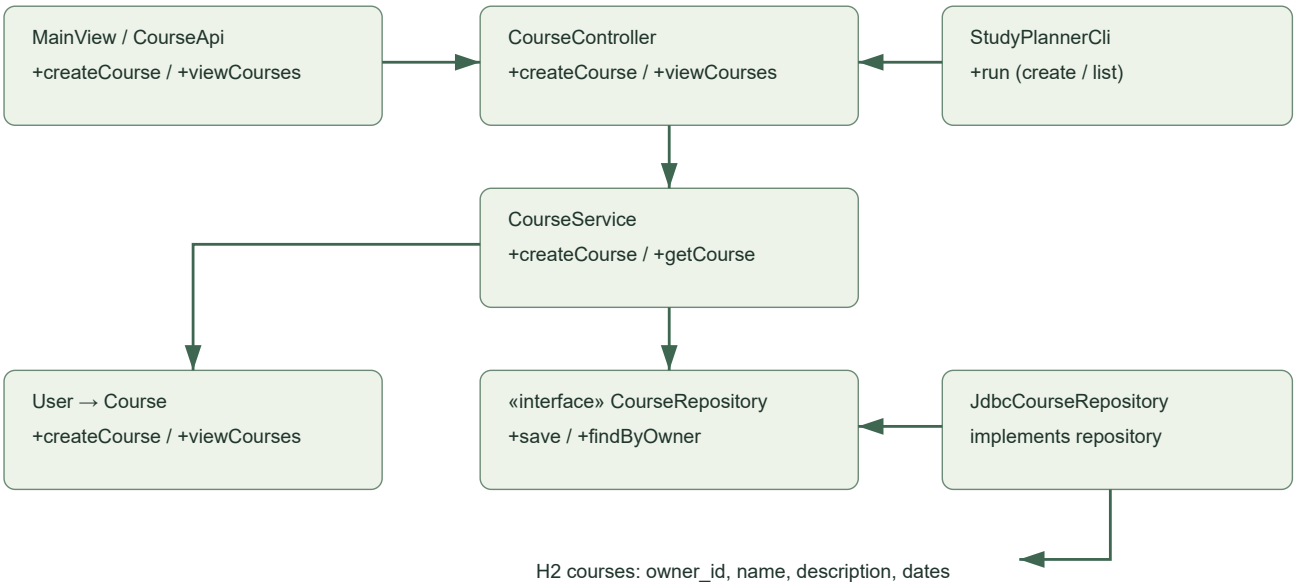
3. Implemented design patterns and updated diagrams

The five Stage 1 patterns remain planned, not implemented: Strategy (grading algorithm selection), Observer (calendar synchronization), Facade (AI subsystem boundary), Singleton (shared AIEngine), and Factory (study-resource creation). There is no appropriate reason to force these AI/calendar patterns into deterministic course creation. The final report must replace these status notes with

actual source files, roles, methods, and explanations as those features are built. Framework-managed singleton beans alone do not demonstrate the planned AIEngine Singleton.

The implemented architecture adds a Repository abstraction: CourseService depends on CourseRepository; JdbcCourseRepository supplies durable storage without coupling the domain to JDBC. This addresses a real persistence boundary but is additional architecture, not a replacement claim for any of the five course patterns. No new counted pattern is claimed in this increment.

Current implementation class diagram (F01 subset; not the final complete-system UML):



Editable Mermaid class, sequence, and use-case scope diagrams are in `documentation/diagrams/stage2-f01.md`. They supplement, rather than overwrite, the Stage 1 diagrams. The creation sequence now includes explicit HTTP clients, security, application service, and persistent repository. The upload branch of SD01 remains future F02 work.

4. Changes and rationale

Original design	Implemented change	Reason / impact
F01 / UC-01 / SD01: UI → CourseController → User → Course.	MainView → CourseApi → CourseController → CourseService → User.createCourse → CourseRepository/JDBC.	Preserves creation and ownership while making the already selected separate frontend/backend communicate through a real REST boundary and adding transactions/persistence. Affects creation/list/read sequence; no replacement of user need or AI patterns.

User holds courses; User.createCourse/viewCourses.	User creates immutable owned courses and queries repository; owner ID is derived from the authenticated username, rather than storing a mutable object collection.	Courses must survive restarts and be consistently scoped to the student. A username change creates a different owner identity; full user/account persistence is future work. Stable UUID derivation is local-account identity, not authentication.
Authenticated student is a precondition; registration not specified as a counted feature.	One locally configured account verified by Spring Security; sign-in/sign-out supports GUI access; API and CLI use Basic + CSRF.	Enforces missing/invalid-authentication paths without pretending a full identity-management feature exists. Credentials are configured outside Git, hashed for the in-memory provider, and never accepted as owner IDs.
F01 “create and manage,” specifically four inputs and newly displayed course.	Creation, owned list, and owned read implemented. No edit/delete. All four fields required, explicit length limits, inclusive date order.	Bounded interpretation of the documented first workflow; constraints make errors deterministic and protect storage. No original F02–F10 feature is removed or replaced.
GUI and CLI required overall; CLI design unspecified for F01.	Java source-file CLI uses exactly the same secured REST API.	Provides a real second interface without duplicating domain validation; supports incremental CLI growth for future agent functionality. F01 CLI alone does not satisfy future agent CLI requirements.

5. AI tools and collaboration log

Tool/platform and model	Use	Total tokens / money
OpenAI Codex coding agent (GPT-6-based; exact runtime model ID not exposed in this session).	Read the Stage 1 and Stage 2 PDFs using temporary Apache PDFBox; traced F01, UC-01, and SD01; implemented Java GUI/API/domain/storage/CLI, tests, documentation, and this report.	Not available from the agent's tools. Owner must insert the actual usage and billing totals from the development platform before submission. Do not substitute an estimate or zero.

Representative interaction: Human: implement course creation F01 using the documentation, follow Stage 2 restrictions, update the report, and explain the result. Agent: identified four required fields, authentication/error conditions, persistent ownership, GUI/CLI parity, and unimplemented future requirements; implemented and checked the vertical flow. Human review and understanding remain required. The human must verify behavior and cost records and record demonstrations before submission. No LLM is invoked by F01 itself.

6. Development and verification summary

Verification completed for F01: `backend/mvnw.cmd clean verify` (followed by verify after correcting BCrypt configuration) passed six tests with zero failures/errors; the frontend production package passed. Backend HTTP integration tests cover successful create/list/read, trimmed text, persistence, field limits, missing/blank values, invalid and reversed dates, same-day courses, invalid JSON, unauthenticated/wrong-password requests, CSRF rejection, and cross-owner isolation. Tests use a separate in-memory database. The real Chrome browser workflow and Java CLI both created courses against an isolated file-backed backend. Both courses survived a real backend restart. GUI checks passed for failed sign-in, cancel, required-field and reversed-date validation, successful creation, refresh, reload/re-authentication, sign-out, backend-unavailable error, and mobile layout without horizontal overflow. Screenshots were visually reviewed. Verification servers use ports 8090/8091, a unique target database, random ephemeral credentials, and are stopped afterward. No submitted video is claimed.

Repeatable runtime checks: `scripts/verify-f01.ps1` launches the packaged apps and drives the CLI; `scripts/verify-f01-browser.mjs` checks the real Vaadin UI. The temporary browser driver is installed under ignored `frontend/target/browser-tools`; see the root README. Test fixtures live only in ignored target files and are not mixed into the student's database.

Important defect found: the initial BCrypt provider configuration lacked the password encoding ID required by Spring's default delegating encoder. A real HTTP Basic integration test exposed it; adding the `{bcrypt}` prefix corrected it. The earlier starter was only compiled before UI verification; this increment requires an actual browser workflow and CLI/API checks. No test passwords are production credentials.

Toronto reporting period	Required	Verified local evidence and outstanding work
October 20–26, 2026	≥5 meaningful code commits	No qualifying commits found in the inspected local history. Owner must check remote evidence. Do not backdate or fabricate activity.
October 27–November 2, 2026	≥5 meaningful code commits	Observed starter commit: a0db921 — frontend/backend initialization. This F01 increment is not committed/pushed by the agent. Owner must add real development commits and links as work progresses.
November 3–9, 2026	≥5 meaningful code commits	Future period at this report date; pending.

The assignment deadline is November 10, 2026. The weekly minimum applies independently to each period. Pending work includes all remaining features, five implemented patterns, agent/LLM behavior and CLI access, complete-system UML, real usage costs, human review, recorded videos, and commit/push evidence. The current report does not claim these requirements are met.

Secrets protection: root `.gitignore` excludes local environment/configuration and database files before introducing credentials. `backend/config.local.properties.example` contains only placeholders; Spring explicitly imports the ignored local properties file. The README explains setup. No LLM API key is requested or stored for F01.

7. Demonstration index

Feature	Recording / timestamp	Required demonstration
F01 Course Creation (deterministic)	Pending human recording and accessible URL/timestamp.	Show GUI sign-in, empty dashboard, create form, required fields, invalid date range, cancel, valid creation and dashboard output. Refresh/restart to show saved data. Show failed sign-in and API-unavailable behavior. Show CLI list and create, invalid-input failure, and the GUI reflecting the CLI-created course. Explain that no AI is needed for this feature. Keep credentials and local files out of the recording.
F02–F10 and UC-11	Not yet implemented; recordings pending.	Add clearly labeled working segments as each feature is completed. Collective recordings must show GUI and CLI, not screenshots alone.

Browser verification artifacts are development evidence only and do not replace the assignment's video demonstrations. Generate the PDF from this HTML using the browser Print → Save as PDF after each report update.